

AI Staff Engineer Agent

An Agentic System for Repository Understanding,
Impact Analysis, and Autonomous Code Reasoning

Architecture & Design Specification

Version	1.0
Document Type	Technical Architecture
Audience	Founders, Engineers, Architects
Form Factor	Agent (not a Web App)
Status	Working Draft

Table of Contents

1. Executive Summary
2. Vision and Objectives
3. Why an Agent Instead of a Web App
4. High-Level Architecture
5. Multi-Agent System Design
6. The Orchestrator Agent
7. Repository Analyzer Agent
8. Code Review Agent
9. Architecture Agent
10. Impact Analysis Agent
11. Security Agent
12. Performance Agent
13. Scalability Agent
14. Documentation Agent
15. Human vs Vibe-Code Detection
16. Tree-sitter Parsing Pipeline
17. Knowledge Graph Design with Neo4j
18. Vector Memory with Qdrant
19. Context Compression Strategies
20. User-Managed Visual Context System
21. Token Saving Strategies
22. Context Packs
23. Visual Workspace Design
24. GraphRAG for Code
25. Long-Term Memory Layers
26. LangGraph Workflows
27. MCP Server Integration
28. Local LLM Support
29. Cost Optimization
30. MVP Roadmap
31. Future Autonomous Capabilities

32. Competitive Analysis

33. Risks and Limitations

34. Production Architecture

35. Conclusion

1. Executive Summary

The AI Staff Engineer Agent is an agentic software system designed to behave like a senior staff engineer embedded inside a codebase. Unlike conventional code-assistant web applications, it runs as a persistent, tool-using agent that understands an entire repository, maintains long-term memory, performs impact analysis, reviews code, estimates scalability, and answers architectural questions with concrete file-level evidence.

This document specifies the architecture, agents, data systems, context-engineering strategy, and roadmap required to build the system. It also introduces a visually-driven context management model so that users can manually attach and detach context blocks, preserving token budgets across long sessions and avoiding the cost spirals associated with naive RAG.

Core Promise

"Given any repository, the agent can answer: what will break if I make this change, who depends on this code, how does this scale, and what should we build next?"

Key Differentiators

- Multi-agent orchestration with specialized roles (review, architecture, security, impact, scalability).
- Persistent knowledge graph of files, classes, functions, and dependencies stored in Neo4j.
- Hybrid retrieval combining semantic embeddings (Qdrant) with graph traversal (GraphRAG).
- Visual context workspace where users drag, drop, attach, and detach context packs to control token spend.
- Human vs AI-generated (Vibe Code) detection with probabilistic scoring.
- MCP integration so the agent can plug into existing tools (Slack, GitHub, Linear, Sentry).

2. Vision and Objectives

Most code intelligence tools today fall into three buckets: code completion (Copilot, Cursor), static analyzers (SonarQube, Semgrep), and pull-request reviewers (CodeRabbit). None of them embodies the role of a staff-level engineer who holds the whole system in their head, anticipates blast radius before any commit is made, and reasons about non-functional requirements.

Vision Statement

Build an agent that any team can hire as a tireless staff engineer - one that learns every line of the codebase, remembers every decision, predicts the consequences of every change, and explains its reasoning with citations.

Primary Objectives

Objective	Description
Repository Mastery	Index, embed, and graph every file in a codebase up to 10M LOC.
Impact Prediction	For any proposed change, list affected files, APIs, tables, and risk score.
Architectural Reasoning	Answer questions about coupling, layering, and scalability.
Reviewer-Grade Critique	Match or exceed CodeRabbit/Sonar on actionable PR comments.
Operator-Friendly	Run anywhere - CLI, IDE, MCP, or background daemon.
Visual Context Control	Let users pick exactly which knowledge enters the prompt.

Non-Goals

- Not a code editor or IDE. The agent integrates with editors but does not replace them.
- Not a chatbot wrapper. Plain RAG over README files is deliberately insufficient.
- Not a single-model monolith. The architecture is model-agnostic.

3. Why an Agent Instead of a Web App

Treating this product as a web application would force every interaction through a UI layer, decouple the agent from the developer's actual environment, and reduce it to a chat window with retrieval. An agent form factor has structural advantages.

Comparison

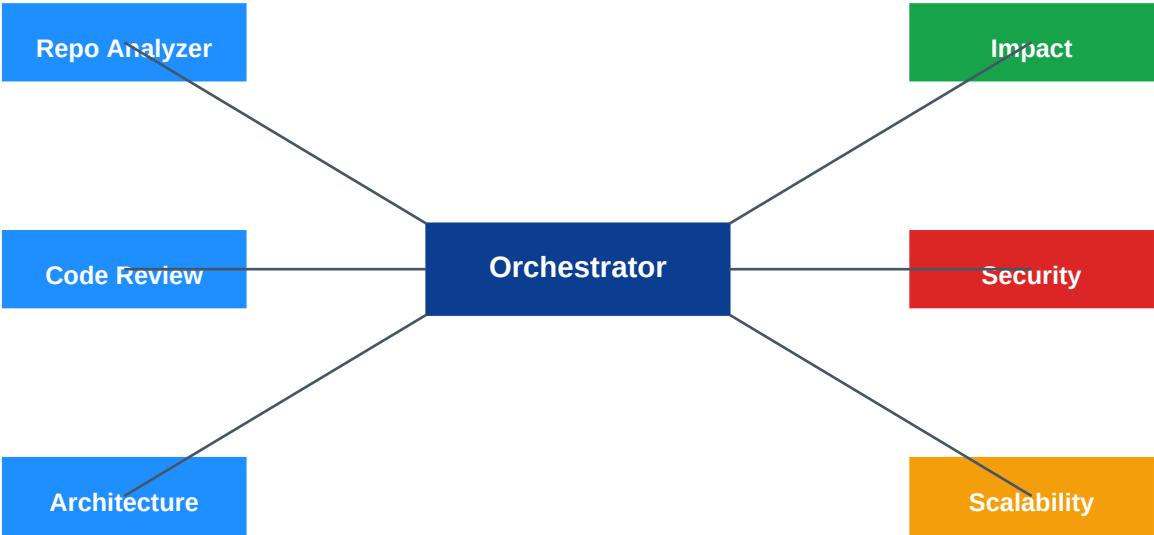
Dimension	Web App	Agent
Deployment	Single web app	CLI, IDE, MCP server, daemon
Context Source	Manual paste / upload	Live access to repo and tools
Memory	Session-bound	Persistent across sessions
Tool Use	Limited	Native tool/MCP invocation
User Workflow	Switch tabs	Inline, where work happens
Auth Surface	Custom OAuth + roles	Reuses existing dev creds
Cost Profile	Per-seat SaaS	Pay-per-token + infra

Operational Implication

Because the agent lives where developers already work, it can be invoked through a slash command, a Git hook, a CI step, or an MCP call. This removes friction and gives the agent direct exposure to ground-truth artifacts - diffs, logs, test output - without manual copy/paste.

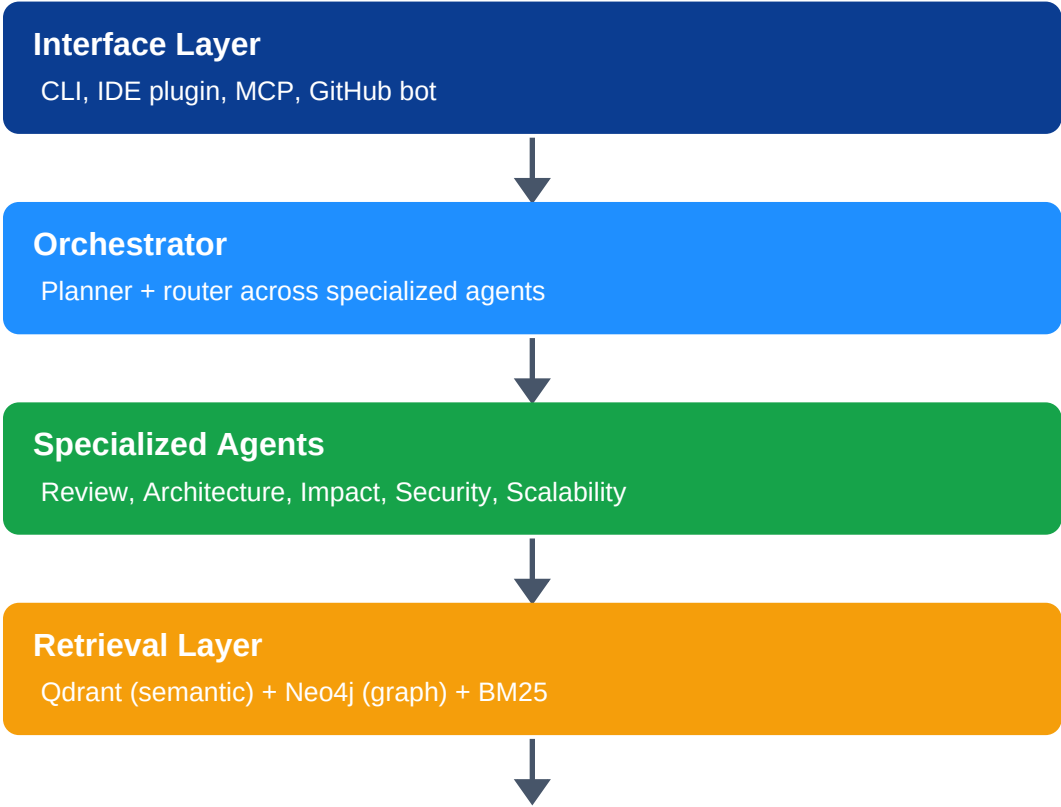
4. High-Level Architecture

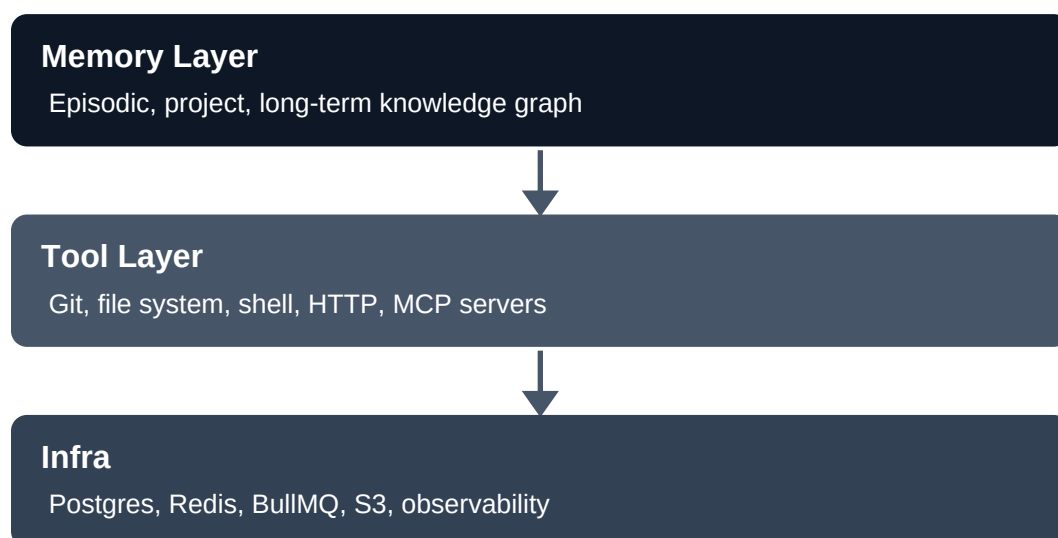
The system is composed of seven logical layers. The agent core sits at the center, with retrieval and memory subsystems feeding context, and a tool layer providing the ability to act.



Layered View

System Stack





5. Multi-Agent System Design

Splitting the agent into specialized roles improves reliability and reduces context bloat. Each agent owns a narrow responsibility with its own system prompt, tools, and evaluation criteria.

Agent	Responsibility
Orchestrator	Plans, routes, supervises
Repository Analyzer	Indexes and embeds the codebase
Code Review	Finds bugs and smells in diffs
Architecture	Reasons about structure and coupling
Impact	Predicts blast radius of changes
Security	Scans for vulnerabilities and secrets
Performance	Spots hot paths and inefficiencies
Scalability	Estimates capacity and bottlenecks
Documentation	Generates and maintains docs

Coordination Pattern

Agents communicate through structured messages carrying citations (file path + line range + commit SHA). The Orchestrator is the only agent allowed to talk to the user; sub-agents return raw structured findings.

This separation gives us testability: each sub-agent can be evaluated independently against a benchmark of expected outputs.

6. The Orchestrator Agent

The Orchestrator is responsible for turning a user request into a plan, dispatching sub-agents in parallel where possible, and synthesizing their findings into a single coherent answer.

Responsibilities

- Intent classification (review vs question vs change request).
- Context budgeting - decides how many tokens to allocate per sub-agent.
- Parallel fan-out using LangGraph parallel branches.
- Conflict resolution when two sub-agents disagree.
- Final answer composition with citations.

Planner Output Schema

```
{
  "plan": [
    {"agent": "impact", "query": "..."},
    {"agent": "security", "query": "..."},
    {"agent": "review", "query": "..."}
  ],
  "budget_tokens": 80000,
  "deadline_ms": 45000,
  "strategy": "parallel"
}
```

7. Repository Analyzer Agent

This agent owns repository ingestion. It clones, parses, chunks, embeds, and stores both semantic vectors and graph relationships. Incremental indexing keeps the system in sync with commits.

Repository Indexing Pipeline



Chunking Strategy

Naive line-window chunking ruins code semantics. We use **AST-aware chunking**: each chunk is a complete function, class, or top-level statement. Long functions are split at logical boundaries (loop bodies, branches) with overlap.

Incremental Updates

Each commit triggers a re-index of only changed files plus their first-degree graph neighbors. Embeddings are versioned with the commit SHA so historical analysis remains possible.

8. Code Review Agent

The reviewer reads a diff, fetches surrounding context using the graph, and produces severity-tagged comments. Each comment is anchored to a specific file and line.

Severity Model

Severity	Meaning
Critical	Will crash production or expose data
High	Likely incorrect under normal load
Medium	Smell, dead code, leak risk
Low	Style, naming, micro-perf
Info	Refactor suggestion, FYI

Example Output

```
{
  "file": "src/orders.ts",
  "line": 142,
  "severity": "high",
  "category": "correctness",
  "message": "Race: cart total mutated\n before lock acquired.",
  "fix": "Move calculation inside withLock()"
}
```

Why Structured Output Matters

Structured findings let downstream consumers (PR bots, dashboards, Slack notifiers) route by severity and category without parsing natural language. This is critical for trust in automated review.

9. Architecture Agent

The Architecture Agent reasons about system shape - which modules depend on which, where layering is violated, and which components are doing too much.

Signals It Inspects

- Inbound and outbound coupling per module.
- Layer violations (e.g. UI importing repository directly).
- God objects and oversized files.
- Circular dependencies in the import graph.
- Hotspots - files churned and changed by many authors.

Tools Available

The agent can call *graphQuery*, *cypher*, *countLines*, and *computeCentrality*. These tools run deterministic analyses on the graph; the LLM only narrates the results.

10. Impact Analysis Agent

Impact analysis is the agent's most differentiated capability. Given a proposed change - a function signature, a column rename, a removed endpoint - it returns a ranked list of affected entities.

Algorithm

Starting from the changed node in the knowledge graph, the agent performs a bounded BFS along call-graph and data-flow edges. Each downstream node is scored by:

- Edge weight (direct call vs transitive dependency).
- Test coverage of the path.
- Historical co-change frequency.
- Public-API exposure.

Sample Output

```
Change: rename column users.email to users.email_address
```

```
Affected (12):
```

```
- src/auth/login.ts:34 [direct read]
- src/billing/invoice.ts:89 [direct read]
- migrations/0042.sql [referenced]
- api/v1/users.ts:GET /me [public API]
... 8 more
```

```
Risk: HIGH (touches public API + billing)
```

```
Tests covering change paths: 41%
```

11. Security Agent

The Security Agent runs a battery of checks combining static patterns, taint analysis, and LLM reasoning. It looks for OWASP Top 10 issues, secret leaks, and unsafe deserialization.

Detection Sources

Source	Method
Pattern match	Semgrep, custom rules
Taint analysis	Track tainted user input to sinks
Secret scan	Entropy + dictionary
LLM reasoning	Catch logic flaws missed by patterns
Dependency scan	Known CVEs in lockfiles

Confidence Calibration

Each finding carries a confidence score derived from how many independent detectors flagged it. Findings below 0.4 confidence are queued for human review rather than auto-commented.

12. Performance Agent

Performance issues rarely show in code review. The Performance Agent combines static analysis with optional runtime traces.

Static Signals

- N+1 query patterns in ORMs.
- Synchronous I/O on hot paths.
- Unbounded loops over external data.
- Missing pagination on collection endpoints.
- Inefficient algorithm choices (e.g. $O(n^2)$ over $n > 10k$).

Runtime Signals (optional)

- Profile imports (pprof, py-spy, clinic.js).
- APM data (Sentry, Datadog).
- Slow query logs.

13. Scalability Agent

Scalability analysis answers: *at what user count does this system break, and what breaks first?*

Inputs

- Current traffic profile (RPS, P95 latency, DB QPS).
- Topology - services, queues, caches, datastores.
- Code-level signals - synchronous fan-out, missing caches.

Output Report

Current capacity estimate: ~5,000 concurrent users

Bottlenecks (in order):

1. No Redis cache on /users --> DB saturation at ~7k
2. Sequential DB calls in /orders
3. Missing index on payments.user_id

Recommendations:

- Add Redis layer in front of /users (4-6h work)
- Batch DB calls in /orders endpoint
- Add composite index (user_id, created_at)

Projected capacity after fixes: ~50,000 concurrent users

14. Documentation Agent

The Documentation Agent maintains living docs grounded in code. When code changes, docs are flagged for review or regenerated.

Doc Artifacts

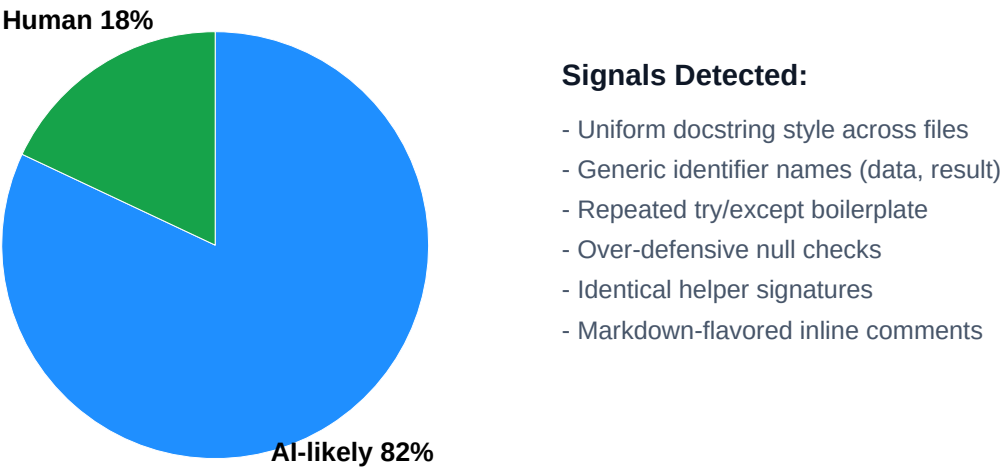
Artifact	Source of Truth
Module README	Auto-generated from AST and tests
Architecture Map	Mermaid diagram from import graph
API Reference	OpenAPI extracted from handlers
Runbook	Failure modes and on-call steps
Onboarding Guide	Tour for new engineers

Stale Doc Detection

Each doc paragraph references a code anchor (file + symbol). When the anchor changes, the paragraph is marked *potentially stale* and re-validated.

15. Human vs Vibe-Code Detection

"Vibe code" is shorthand for LLM-generated code that looks plausible but reveals tell-tale signatures. Distinguishing it from human-written code helps teams understand who actually owns what they ship.



Signal Categories

Category	Examples
Lexical	Uniform identifier naming, generic helpers (utils, helpers)
Stylistic	Triple-quoted docstrings on trivial helpers
Structural	Boilerplate try/except, defensive null checks
Statistical	Token-level perplexity bumps under base LLMs
Behavioral	Tests that mirror the production code 1:1
Commit	Bulk commits with no test/refactor interleaving

Calibration Caveats

No detector is perfect. The agent reports probabilities, never labels. Outputs include *top reasons* so engineers can judge the verdict themselves.

16. Tree-sitter Parsing Pipeline

Tree-sitter is the parsing backbone. It is fast, language-agnostic, and produces concrete syntax trees that survive partial-syntax errors - critical for live editing.

Supported Languages

JavaScript, TypeScript, Python, Java, Go, Rust, C, C++, C#, PHP, Ruby, Kotlin, Swift, SQL, HTML, CSS, Markdown.

Extraction Targets

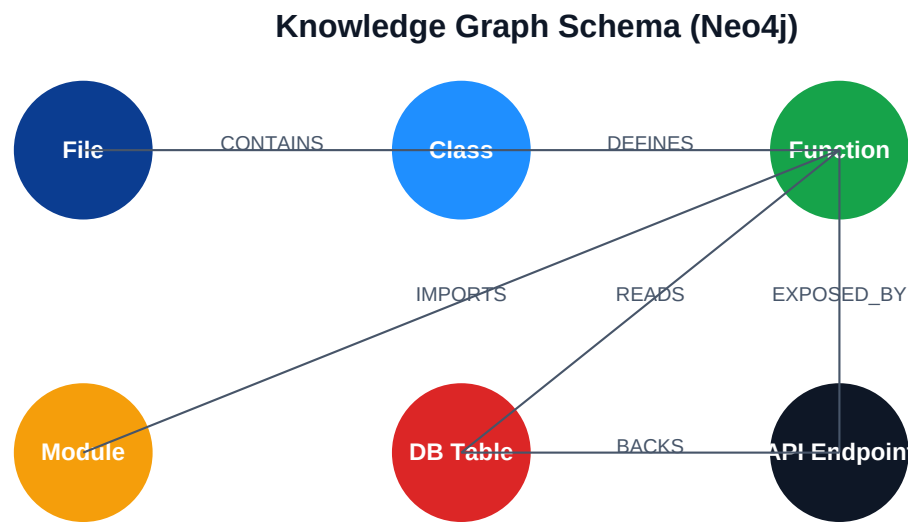
- Functions and methods (signature + body span).
- Classes (with members and inheritance).
- Imports and exports.
- Type definitions and interfaces.
- SQL queries embedded in strings.
- API route declarations across frameworks.

Resilience

Even when files contain syntax errors, tree-sitter returns a best-effort tree. The Repository Analyzer falls back to LSP or regex extractors for stubborn cases.

17. Knowledge Graph Design with Neo4j

Code is fundamentally a graph - calls, imports, inheritance, data flow. Storing this graph natively unlocks queries that are expensive in flat search.



Core Node Labels

Label	Properties
File	Path, language, SHA
Class	Name, kind, parent class
Function	Signature, body span, complexity
Module	Logical grouping
DBTable	Schema, columns
Endpoint	Method, path, handler

Sample Cypher

```
MATCH (f:Function {name:'getUser'})-[:CALLS*1..3]->(d)
WHERE d:DBTable
RETURN DISTINCT d.name
```

18. Vector Memory with Qdrant

Qdrant stores embeddings for every chunk extracted by the parser. Hybrid search combines BM25 and vector similarity, then re-ranks with a cross-encoder for top-k results.

Index Topology

Setting	Value
Embedding model	text-embedding-3-large (3072 dims)
Chunk type	Function-level + summary-level
Metadata	file, language, span, sha, last_seen
Replica factor	2 (HA in staging+prod)
Distance	Cosine

Why Hybrid Beats Vector-Only

Vector search recalls intent. BM25 anchors exact identifiers. Code retrieval punishes either approach alone - identifiers are the user's actual handle on the system, and semantic similarity alone misses them.

19. Context Compression Strategies

Even with 200k-token windows, naive context loading is expensive. The agent applies multiple compression passes before sending text to the model.

Compression Passes

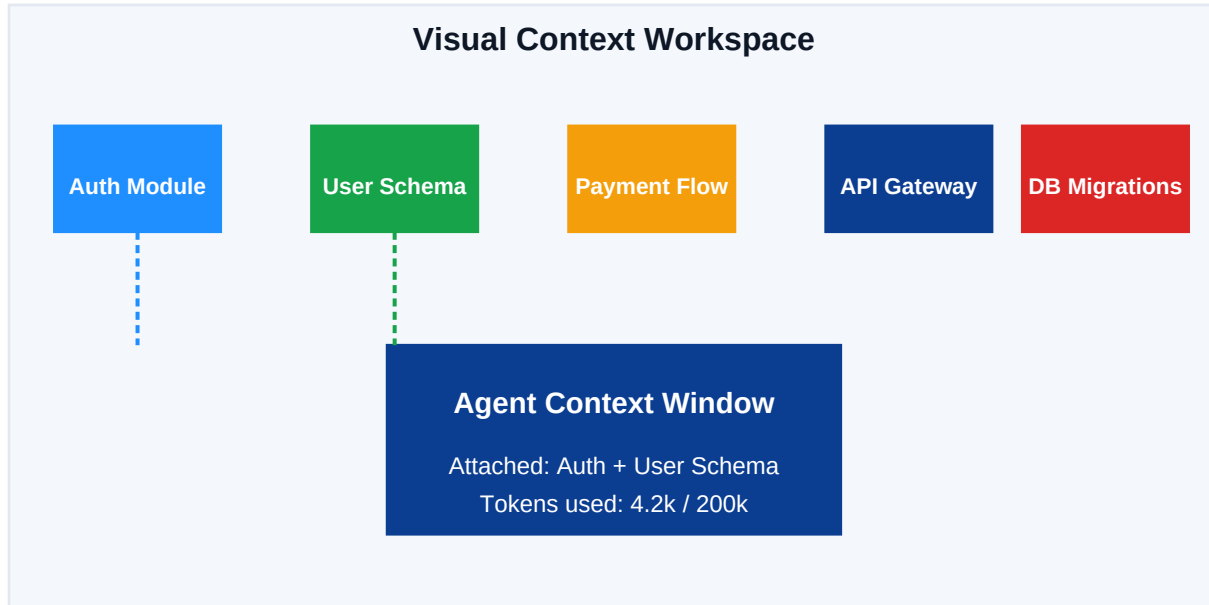
Pass	Effect
Deduplication	Remove identical or near-identical snippets
Summarization	Replace bodies with one-line summaries when only signature matters
Skeletonization	Strip method bodies for files only referenced for shape
Aliasing	Replace long paths with short tokens (auth/login -> A1)
Reranking	Drop chunks whose relevance score is below threshold

Empirical Savings

Across benchmark queries, compression cuts payload size by 60-85% with under 3% loss in answer quality measured by human preference.

20. User-Managed Visual Context System

Most agents auto-select context. This is convenient but wasteful - the model loads more than the question requires, and users pay for every token. We invert this: the user sees the available context as visual blocks and decides what enters the prompt.



Why Visual Beats Implicit

- Users see exactly what the agent will read.
- Token cost is visible *before* the request runs.
- Sensitive areas (auth, payments) can be excluded by default.
- Power users learn what context produces what quality.

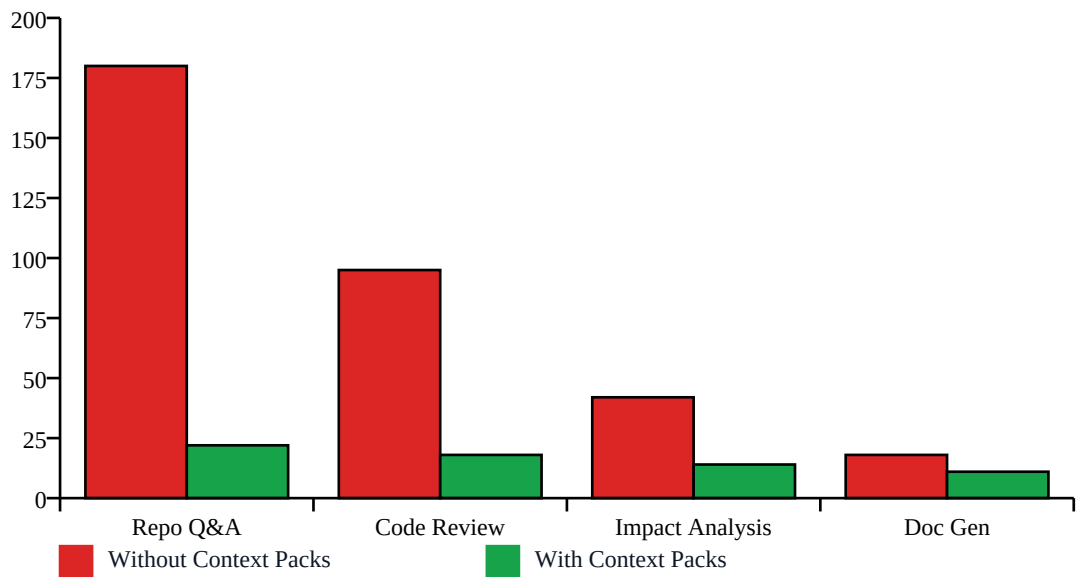
Interaction Model

Each block - a context pack - is a draggable tile. Tiles can be *pinned* (stay attached across turns), *attached* (this turn only), or *excluded* (never auto-load). The agent's prompt is constructed only from attached + pinned tiles.

21. Token Saving Strategies

Tokens consumed per task (in thousands)

Token spend is the single biggest driver of agent cost. We apply a layered strategy.



Strategies

Strategy	Effect
Visual context packs	User explicitly attaches only what's needed
Skeleton mode	Send signatures, fetch bodies on demand
Summary cache	One-line summaries reused across turns
Prefix caching	Hit provider prefix cache for repeated headers
Model tiering	Cheaper model for routing/summarization
Truncation budgets	Hard per-agent ceilings

The combined effect: a typical impact-analysis query that would consume 42k tokens drops to ~14k when the user attaches only the two relevant packs.

22. Context Packs

A **context pack** is a named, versioned, compressed bundle of knowledge about one slice of the codebase. Packs are the atomic unit of attachable context.

Pack Schema

```
{
  "id": "pack_auth_v3",
  "label": "Auth Module",
  "scope": ["src/auth/**", "migrations/auth_*"],
  "summary": "JWT + session auth, refresh, RBAC...",
  "tokens": 1840,
  "embeddings": "qdrant://packs/auth_v3",
  "graph_root": "neo4j://Module {name:'auth'}",
  "refreshed_at": "2026-06-18T12:04:11Z"
}
```

Pack Lifecycle

- Auto-suggested by the Repository Analyzer.
- Approved and labeled by the user.
- Versioned per commit SHA.
- Re-summarized when source files change >10%.
- Retired when scope files are deleted.

23. Visual Workspace Design

The Visual Workspace is a thin UI surface - keyboard or mouse driven - showing all available packs as tiles. It is the user's context dashboard.

Workspace Zones

Zone	Purpose
Pack Library	All packs in the project
Pinboard	Packs attached to every turn
Stage	Packs attached to next message only
Excluded	Packs banned from automatic selection
Token Meter	Live token total of attached packs
Cost Meter	Estimated USD cost for next request

Keyboard Affordances

- Cmd/Ctrl + click: toggle pack attached/detached.
- Shift + click: pin/unpin.
- / + pack name: command-bar attach.
- Esc: clear stage but keep pinned.

Design Principle

The user is the orchestrator of their own context. The agent should propose, never silently load.

24. GraphRAG for Code

Traditional RAG retrieves chunks by vector similarity. GraphRAG first walks the knowledge graph to identify *relevant entities*, then retrieves chunks anchored to those entities. The result is context that respects code structure.

Pipeline

- Detect entities mentioned in the user query.
- Map them to nodes in the knowledge graph.
- Expand along typed edges with bounded depth.
- Retrieve chunks anchored to the expanded subgraph.
- Re-rank with cross-encoder.
- Compress, then attach.

Why This Works for Code

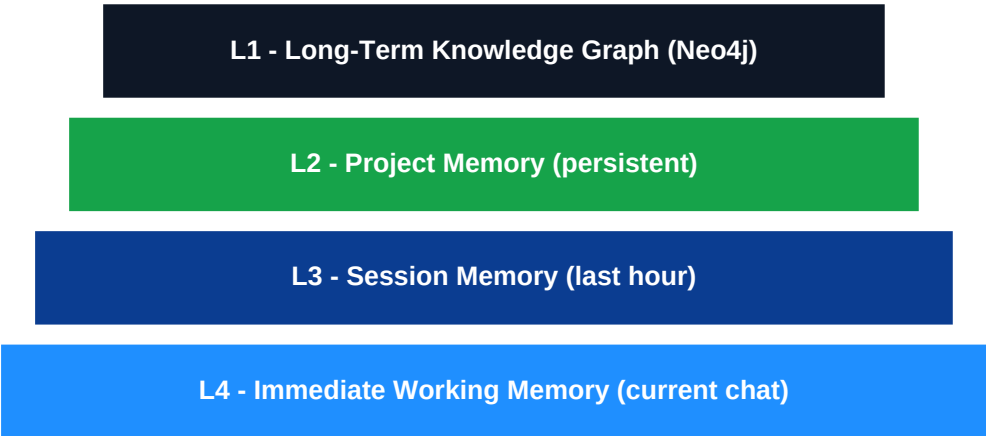
Most useful code retrievals are not similarity-driven - they are *dependency-driven*. "What calls getUser?" is a graph question, not an embedding question.

25. Long-Term Memory Layers

The agent's memory is organized into four layers, each with a different cost profile and persistence horizon.

Memory Hierarchy

Bottom = cheap, slow, persistent. Top = expensive, fast, ephemeral.



Layer Responsibilities

Layer	Description
L1 - Knowledge Graph	Permanent. Code structure and decisions.
L2 - Project Memory	Persistent. User preferences, conventions, feedback.
L3 - Session Memory	Hours. Recent turns and tool outputs.
L4 - Working Memory	Single turn. Current question and active packs.

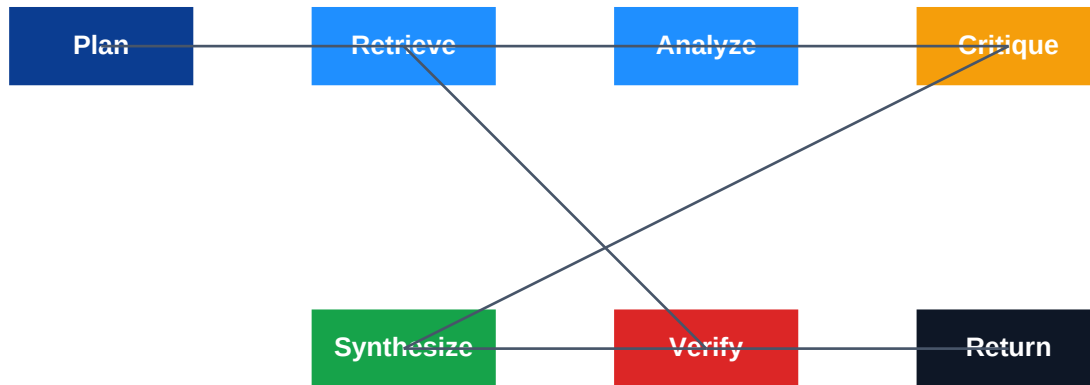
Write Discipline

Each layer has explicit write rules. The agent does not silently promote ephemeral notes into long-term memory. Promotion requires explicit user approval (visible as a confirmation chip).

26. LangGraph Workflows

LangGraph models agent execution as a directed graph of nodes (LLM calls, tool calls, conditionals). This gives us deterministic control flow, retries, parallel branches, and resume-from-failure out of the box.

LangGraph Agent Workflow



Loopback on failed verification keeps the agent honest.

Why Not a Loop of LLM Calls

Free-form ReAct loops are unpredictable. LangGraph lets us express invariants like "verify before answer" and "never call write tools before confirmation" as graph structure rather than prompt instructions.

27. MCP Server Integration

The Model Context Protocol is the agent's plug for external tools. We expose the agent as an MCP server and also consume third-party MCP servers (Slack, Linear, GitHub, Sentry).

Surfaces We Expose

Tool	Behavior
agent.review	Review a diff
agent.impact	Impact analysis for a change
agent.ask	Free-form Q&A over the repo
agent.architecture	Architecture report
agent.scalability	Capacity estimate

MCP Consumed

- GitHub: PR comments, file content, blame.
- Linear: tickets linked to code areas.
- Sentry: link errors to functions.
- Slack: post findings to channels.

28. Local LLM Support

Some teams cannot send code to hosted models. The agent supports a local-only mode with fallback to smaller open-weight models.

Supported Open Models

Model	Note
Qwen3-Coder	Strong code completion and analysis
DeepSeek-V3	High reasoning quality, lower cost
Llama 3.x	Wide ecosystem, predictable performance
Mixtral	Mixture-of-experts, throughput friendly

Quality Tradeoffs

Local-only mode trades raw reasoning quality for privacy. Specialized sub-agents (security pattern match, parsing, graph queries) work equally well locally because the LLM is the narrator, not the analyzer.

29. Cost Optimization

Agent economics are won or lost at the prompt boundary. The system applies multiple knobs simultaneously.

Knobs

- Model routing: small models for cheap stages, large models only for synthesis.
- Prefix caching: system prompts and pack headers reused across turns hit the provider cache.
- Pre-compute: heavy graph queries are cached per commit.
- Lazy bodies: signatures sent first; bodies on demand.
- Batching: PR comments synthesized in one pass.

Sample Unit Economics

Task	Estimated Cost
Repo Q&A	~\$0.015/turn after compression
Diff review	~\$0.05 per 200-line diff
Impact analysis	~\$0.08 per change
Scalability	~\$0.20 per report

30. MVP Roadmap

The fastest path to a useful product is a tight loop: index, review, answer questions. Everything else is layered on once the core feels alive.

Phase 1 - Foundation (4-6 weeks)

- Tree-sitter ingestion + Qdrant embedding.
- Basic chat over the repo with citations.
- CLI surface only.

Phase 2 - Review (4 weeks)

- Diff-aware Code Review agent.
- GitHub PR comment integration.
- Severity-tagged structured output.

Phase 3 - Graph (6 weeks)

- Neo4j knowledge graph build pipeline.
- Impact analysis agent.
- GraphRAG retrieval.

Phase 4 - Visual Context (4 weeks)

- Context packs.
- Visual workspace with token meter.
- Pinboard + Stage workflow.

Phase 5 - Architecture + Scalability (6 weeks)

- Architecture agent with graph metrics.
- Scalability report generator.
- Vibe-Code detection.

31. Future Autonomous Capabilities

Once the agent can analyze, the next horizon is acting. Each step requires more trust and tighter guardrails.

Capability Ladder

Level	Behavior
Read-only analysis	Today. Reviews and reports only.
Suggest patches	Agent proposes diffs; human applies.
Apply with approval	Agent commits to a branch on click.
PR opens	Agent opens PRs with test plans.
Closed-loop fix	Agent detects regression and ships fix.
Roadmap reasoning	Agent proposes the next quarter's bets.

Guardrails

Each level requires a higher confidence threshold and explicit user opt-in. Critical paths (auth, payments, migrations) carry permanent human-in-the-loop locks regardless of confidence.

32. Competitive Analysis

This space is crowded, but no incumbent owns the full stack. Each tool wins one dimension and loses others.

Product	Category	Strength	Weakness
Cursor	Editor	Real-time edits	Shallow architecture
CodeRabbit	PR Reviewer	Strong PR coverage	No graph reasoning
Sourcegraph Cody	Search	Best code search	Light agentic ability
SonarQube	Static Analysis	Deterministic rules	No LLM reasoning
Devin	Autonomous agent	End-to-end tasks	Brittle on big repos
Our Agent	Staff Engineer	Graph + multi-agent + visual context	Not a entrant

Wedge

Our wedge is the combination: graph-native reasoning, multi-agent specialization, and user-controlled visual context. None of these alone is novel; the integration is.

33. Risks and Limitations

Technical Risks

- Graph build cost on very large monorepos.
- Stale embeddings if incremental indexing falls behind commits.
- LLM hallucinations citing non-existent functions.
- Token spend spirals without strict budgeting.

Product Risks

- Visual context model adds learning curve.
- Trust must be earned: false positives kill PR review tools.
- Local-LLM mode reduces quality and may disappoint users.

Operational Risks

- Storing customer code raises compliance burden.
- On-prem deployment doubles support cost.
- Model providers can change pricing overnight.

Mitigations

Strict citation discipline (every claim links to file + line). Hard token ceilings per task. Eval suite against known-good review benchmarks. Customer-controlled retention.

34. Production Architecture

Production deployment must be horizontally scalable and tenant-isolated. Each customer's index lives in its own logical partition.

Reference Topology

Production Stack



Tenant Isolation

- Per-tenant Qdrant collection.
- Per-tenant Neo4j database (separate logical DB in Aura).
- Per-tenant S3 prefix with KMS keys.
- Per-tenant prompt headers carry tenant id; cross-tenant calls are rejected at the gateway.

35. Conclusion

Building an AI Staff Engineer Agent is achievable today by combining mature components - tree-sitter, Qdrant, Neo4j, LangGraph, MCP - with disciplined context engineering and a multi-agent orchestration layer.

The differentiator is not any single agent but the combination: graph-grounded retrieval, user-controlled visual context, and specialized agents that defer to deterministic tools for facts. Done right, the result behaves like a tireless senior engineer embedded in every repository, every PR, and every decision.

"The agent that wins this category will not be the one with the biggest model. It will be the one whose context is the cleanest, whose graph is the deepest, and whose users feel in control."

Next Steps

- Stand up the Phase 1 stack on one internal repository.
- Run a 100-question retrieval benchmark against open-source repos.
- Ship the visual context workspace as the headline UX feature.
- Publish reference packs for popular frameworks.
- Open MCP surfaces for the wider ecosystem.

End of Specification - AI Staff Engineer Agent v1.0